CS24301 — THEORY (3.0 CR) MODULE III NOTES BIT MESRA NEP SCHEME

Module 3: Semantic Analysis & Type Systems

SDD, SDTS, Synthesized/Inherited, Type Checking & Symbol Tables

Module III: Semantic Analysis & Intermediate Code Generation — 9-Topic Master Guide

Topic 18: Introduction to Semantic Analysis (Static vs. Dynamic Semantic Checks)

Topic 19: Syntax-Directed Definitions (SDD Attribute Grammars & Dependency Graphs)

Topic 20: Syntax-Directed Translation Schemes (SDTS Embedded Semantic Actions)

Topic 21: SDTS for Declaration Processing & Scoped Symbol Table Design

Topic 22: Three-Address Code (Quadruples, Triples, Indirect Triples & DAG IR)

Topic 23: Types of Attributes (Synthesized vs. Inherited & S-Attributed vs. L-Attributed)

Topic 24: Type Checking for Expressions (Type Equivalence, Inferences & Coercions)

Topic 25: Intermediate Code Generation for Complex Assignment Statements

Topic 26: Translation of Multi-Dimensional Array References (Row/Col Major Math)

Topic 18: Introduction to Semantic Analysis & Static Type Systems

Semantic Analysis constitutes the third major phase of a modern optimizing compiler. While syntax analysis ensures that a program conforms to the formal context-free grammar of the programming language (e.g., verifying matching parentheses, balanced statement delimiters, and correct keyword placement), it is mathematically incapable of enforcing contextual relationships—such as checking whether a variable is declared before its point of use, verifying that array index types match declared array bounds, or validating that operator argument signatures conform to type rules.

Semantic Check Category	Compiler Responsibility & Mechanism	Representative Error Scenarios
1. Static Semantic Checks *(Evaluated at Compile-Time)*	Executed directly by the compiler front-end by traversing the Abstract Syntax Tree (AST) with Symbol Table queries.	• Type mismatch: assigning a string literal to an integer scalar variable. • Undeclared identifier referenced in an arithmetic expression. • Duplicate identifier declared within the same scope block. • Function called with an incorrect number or types of actual arguments. • `break` or `continue` statement used outside of a loop or switch body.
2. Dynamic Semantic Checks *(Evaluated at Run-Time)*	Emitted by the compiler as runtime boundary assertions or trapped by target CPU hardware exceptions during execution.	• Arithmetic division by zero (`x / 0`). • Array index out of bounds (`arr[100]` on array of size 10). • Dereferencing a `NULL` or uninitialized memory pointer. • Stack overflow resulting from unbounded recursive procedure calls. • Dynamic type casting errors (e.g., `ClassCastException` in Java).

Topic 19 & 23: Syntax-Directed Definitions (SDD) & Attribute Grammars

A **Syntax-Directed Definition (SDD)** is a Context-Free Grammar augmented with attributes and semantic rules. An attribute is a quantity associated with a grammar symbol (such as a numerical value, a data type, a memory offset, a symbol table entry pointer, or a generated string of intermediate code).

Formal Classification: Synthesized vs. Inherited Attributes

- **Synthesized Attribute:** An attribute of a non-terminal node A in a parse tree is synthesized if its value is computed exclusively from the attribute values of the **children** of A:

$$A.s = f(Y_1.a, Y_2.a, \ldots, Y_k.a) \quad \text{for production } A \to Y_1 Y_2 \ldots Y_k$$

  Synthesized attributes represent *bottom-up information flow* (from terminal leaves upward toward the parse tree root).
- **Inherited Attribute:** An attribute of a grammar symbol Y_j on the RHS of a production is inherited if its value is computed from the attribute values of the **parent** A and/or the **left siblings** $Y_1, \ldots, Y_{j-1}$:

$$Y_j.i = f(A.a, Y_1.a, \ldots, Y_{j-1}.a) \quad \text{for production } A \to Y_1 \ldots Y_j \ldots Y_k$$

  Inherited attributes represent *top-down and left-to-right information flow* (such as propagating declared data types down to lists of variable identifiers).

19.1 S-Attributed vs. L-Attributed Definitions: Exhaustive Engineering Comparison

Evaluation Dimension	S-Attributed Definitions	L-Attributed Definitions
Permissible Attributes	Utilizes **ONLY Synthesized attributes**. Zero inherited attributes permitted.	Permits both **Synthesized attributes** AND **Inherited attributes** (subject to strict dependency ordering).
Dependency Direction	Dependencies flow strictly upward from child nodes to parent nodes.	Dependencies flow downward from parent to children, and strictly **from left to right** among sibling nodes.
Parse Tree Traversal	Evaluated in a single bottom-up Post-Order traversal of the parse tree.	Evaluated in a single Depth-First, Left-to-Right Pre-Order traversal.
Bottom-Up LR Parsing	Can be evaluated **on-the-fly during LR bottom-up parsing** by executing semantic actions upon reduction. Attribute values are held in an auxiliary parser stack.	Requires special transformations (such as inserting marker non-terminals ϵ-rules) to evaluate on-the-fly in bottom-up LR parsers.
Top-Down LL Parsing	Easily evaluated in LL parsers by passing synthesized attributes up as procedure return values.	**Perfect Natural Match:** Inherited attributes are passed down as function arguments; synthesized attributes returned as results.

Topic 20 & 21: Syntax-Directed Translation Schemes (SDTS) & Declarations

A **Syntax-Directed Translation Scheme (SDTS)** is a context-free grammar where program fragments called **semantic actions** are embedded directly within the right-hand sides of productions inside curly braces `{...}`.

🏛️ Step-by-Step SDTS: Processing Multi-Variable Declarations `int x, y, z;`

The compiler must propagate the base type `int` across an arbitrary sequence of declared variable identifiers:

```
D  → T { L.in = T.type } L
T  → int { T.type = integer }
T  → float { T.type = real }
L  → { L1.in = L.in } L1, id { enter_symbol_table(id.name, L.in) }
L  → id { enter_symbol_table(id.name, L.in) }
```

Complete Annotated Parse Tree Evaluation Walkthrough:

1. The parser reaches the leaf token `int` under non-terminal T. The semantic rule `{T.type = integer}` computes the synthesized attribute $T.\text{type} = \text{integer}$.
2. At the production $D \rightarrow T\ L$, the semantic action `{L.in = T.type}` assigns the inherited attribute $L.\text{in} = \text{integer}$ to child non-terminal L.
3. Node L expands to $L_1, \mathbf{id}_3$ (representing identifier `z`). It assigns $L_1.\text{in} = L.\text{in} = \text{integer}$ and invokes `enter_symbol_table("z", integer)`.
4. Node L_1 expands to $L_2, \mathbf{id}_2$ (representing identifier `y`). It assigns $L_2.\text{in} = L_1.\text{in} = \text{integer}$ and invokes `enter_symbol_table("y", integer)`.
5. Node L_2 expands to the base production $\mathbf{id}_1$ (representing identifier `x`). It executes `enter_symbol_table("x", integer)`.

Topic 22: Three-Address Code (TAC) & Intermediate Representations

Three-Address Code (TAC) is a linearized, machine-independent intermediate language where each statement contains at most one operator and at most three operands of the general form: $x = y\ \mathbf{op}\ z$.

Representation Form	Internal Structural Data Fields	Key Advantages & Tradeoffs
1. Quadruples	Explicit record containing 4 fields: `(op, arg1, arg2, result)`. Intermediate values are assigned explicit temporary variable names (`t1`, `t2`).	• Statements can be reordered freely by optimization passes. • Symbol Table is enlarged to track numerous compiler-generated temporary names.
2. Triples	Explicit record containing 3 fields: `(op, arg1, arg2)`. Results of computations are referenced implicitly by their array statement index `(0)`, `(1)`.	• Saves memory by eliminating temporary variable names. • Code reordering or optimization requires updating all dependent pointer index references.
3. Indirect Triples	Consists of a primary array of pointers indexing into a separate static table of Triples.	• **Best of Both Worlds:** Optimizers reorder, insert, or delete instructions simply by modifying the pointer array without rewriting Triples.

🏛️ Complete Comparative Implementation: Translation of `x = (a + b) * - c + (a + b)`

Index	Quadruple: (op, arg1, arg2, result)	Triple: (op, arg1, arg2)	Indirect Triple Pointer Array
(0)	`(+, a, b, t1)`	`(+, a, b)`	Statement `[0]` → Triple (0)
(1)	`(uminus, c, -, t2)`	`(uminus, c, -)`	Statement `[1]` → Triple (1)
(2)	`(*, t1, t2, t3)`	`(*, (0), (1))`	Statement `[2]` → Triple (2)
(3)	`(+, a, b, t4)`	`(+, a, b)`	Statement `[3]` → Triple (3)
(4)	`(+, t3, t4, t5)`	`(+, (2), (3))`	Statement `[4]` → Triple (4)
(5)	`(=, t5, -, x)`	`(=, x, (4))`	Statement `[5]` → Triple (5)

Topic 24 & 25: Type Checking & Type Conversions

Type Equivalence & Conversion Definitions: - **Name Equivalence:** Two types are considered equivalent if and only if their declared type names are identical. - **Structural Equivalence:** Two types are considered equivalent if they possess identical internal memory layouts, component counts, and field types (even under different type aliases). - **Type Coercion (Implicit Widening):** Automatic conversion performed by the compiler without explicit programmer syntax (e.g., promoting an `int` to a `float` in `float_var = int_var + 2.5`). - **Type Casting (Explicit Narrowing):** Programmer-directed type override (e.g., `(int) float_var`) which may incur precision truncation.

Topic 26: Translation of Multi-Dimensional Array References

While programming languages provide multi-dimensional coordinate arrays $A[i_1, i_2, \ldots, i_k]$, physical memory hardware is strictly a **one-dimensional linear byte array**. The compiler must synthesize Three-Address Code to calculate the linear memory offset at runtime.

Figure 3.1: Linear Memory Storage Conventions for Multi-Dimensional Arrays

26.1 Mathematical Derivation of Multi-Dimensional Array Addresses:

1. One-Dimensional Array ($A[\text{low}..\text{high}]$ with element width w):

$$\text{Address}(A[i]) = \text{base}_A + (i - \text{low}) \times w$$

2. Two-Dimensional Array ($A[l_1..h_1,\ l_2..h_2]$ in Row-Major Order):

$$\text{Address}(A[i_1, i_2]) = \text{base}_A + \Big[(i_1 - l_1) \times n_2 + (i_2 - l_2)\Big] \times w$$

Where $n_2 = (h_2 - l_2 + 1)$ represents the number of elements per row (number of columns).

3. Two-Dimensional Array ($A[l_1..h_1,\ l_2..h_2]$ in Column-Major Order):

$$\text{Address}(A[i_1, i_2]) = \text{base}_A + \Big[(i_2 - l_2) \times n_1 + (i_1 - l_1)\Big] \times w$$

Where $n_1 = (h_1 - l_1 + 1)$ represents the number of elements per column (number of rows).

4. Three-Dimensional Array ($A[l_1..h_1,\ l_2..h_2,\ l_3..h_3]$ in Row-Major Order):

$$\text{Address}(A[i_1, i_2, i_3]) = \text{base}_A + \Big[\big((i_1 - l_1) \times n_2 + (i_2 - l_2)\big) \times n_3 + (i_3 - l_3)\Big] \times w$$

Where $n_2 = (h_2 - l_2 + 1)$ and $n_3 = (h_3 - l_3 + 1)$.

5. General k-Dimensional Row-Major Linear Offset (Horner's Rule):

$$\text{Offset} = \Big(\ldots\big((i_1 - l_1) \cdot n_2 + (i_2 - l_2)\big) \cdot n_3 + \ldots\Big) \cdot n_k + (i_k - l_k)$$

$$\text{Address}(\mathbf{A}[\mathbf{i_1}, \ldots, \mathbf{i_k}]) = \text{base}_\mathbf{A} + \text{Offset} \times \mathbf{w}$$

🏛 Step-by-Step Solved Problem: Generating Three-Address Code for $X = A[i, j] + B[k, l]$

Let A be a 10×20 2D array of integers ($w = 4$ bytes) with 0-based indexing ($n_2 = 20$).
Let B be a 15×30 2D array of integers ($w = 4$ bytes) with 0-based indexing ($n_2 = 30$).

```
// 1. Calculate linear offset for array reference A[i, j]:
t1 = i * 20           ; (i * n2)
t2 = t1 + j           ; (i * n2 + j)
t3 = t2 * 4           ; Byte offset = element_offset * 4
t4 = A[t3]            ; Load value from A at byte offset t3

// 2. Calculate linear offset for array reference B[k, l]:
t5 = k * 30           ; (k * n2)
t6 = t5 + l           ; (k * n2 + l)
t7 = t6 * 4           ; Byte offset = element_offset * 4
t8 = B[t7]            ; Load value from B at byte offset t7

// 3. Perform addition and assign to scalar variable X:
t9 = t4 + t8
X  = t9
```

🧠 M3 Active Recall & 10-Question University Exam Master Bank

Q1. Explain Syntax-Directed Definitions (SDD) and differentiate between S-Attributed and L-Attributed definitions with grammar examples. (10 Marks)

An **SDD** is a context-free grammar augmented with attributes associated with grammar symbols and semantic rules associated with productions.
- **S-Attributed Definitions:** Contain exclusively Synthesized attributes, where a node's value is computed strictly from its children: $A.s = f(Y_1.a, \ldots, Y_k.a)$. They can be evaluated naturally during bottom-up LR parsing on the reduction stack.
Example: $E \to E_1 + T \ \{E.\text{val} = E_1.\text{val} + T.\text{val}\}$.
- **L-Attributed Definitions:** Permit both Synthesized and Inherited attributes, but inherited attributes of a RHS symbol Y_j can only depend on inherited attributes of parent A or attributes of left siblings $Y_1 \ldots Y_{j-1}$. They can be evaluated during top-down LL parsing in a single depth-first pass.
Example: $D \to T \ \{L.\text{in} = T.\text{type}\} \ L$.

Q2. Derive the 2D array row-major address calculation formula and generate Three-Address Code for $A[i, j] = B[i, j] + C[i]$. (10 Marks)

Row-Major 2D Address Formula: $\text{Address}(A[i, j]) = \text{base}_A + [(i - l_1) \cdot n_2 + (j - l_2)] \cdot w$.
Generated Three-Address Code Sequence (Assuming 0-based indexing and width $w = 4$):

```
t1 = i * n2_B
t2 = t1 + j
t3 = t2 * 4
t4 = B[t3]          ; Load B[i, j]
t5 = i * 4
t6 = C[t5]          ; Load C[i]
t7 = t4 + t6        ; Sum
t8 = i * n2_A
t9 = t8 + j
t10 = t9 * 4
A[t10] = t7         ; Store into A[i, j]
```

Q3. Compare Quadruples, Triples, and Indirect Triples representations of Three-Address Code across 4 dimensions. (8 Marks)

1. **Field Count:** Quadruples have 4 fields `(op, arg1, arg2, res)`; Triples have 3 fields `(op, arg1, arg2)`; Indirect Triples use an auxiliary pointer array to index into 3-field Triples.
2. **Temporary Names:** Quadruples generate explicit temporaries (`t1`, `t2`); Triples refer to statement indices `(0)`, `(1)`.
3. **Reordering Overhead:** Quadruples can be reordered freely without rewriting; Triples require rewriting all index pointers; Indirect Triples reorder pointers without touching triple records.
4. **Memory Consumption:** Triples are most compact; Quadruples consume extra symbol table memory.

Q4. What is Type Coercion? How does the compiler handle implicit vs explicit type conversions? (6 Marks)

Type Coercion is the automatic conversion of a value from one data type to another performed by the compiler (e.g., widening an `int` to a `float` in `x = float_val + int_val` by inserting `inttofloat`). **Explicit Type Casting** is programmer-specified (e.g., `(int) float_val`), forcing narrowing conversions that may truncate precision.

Q5. Explain the Scoped Symbol Table implementation for nested blocks in C/C++. (8 Marks)

Nested scopes are represented either via a **Tree of Symbol Tables** (where each child scope points to its parent scope via an `enclosing_scope` pointer) or via a **Scoped Hash Table with Scope Markers**. When entering a new block `{`, a new scope level is pushed onto the scope stack. When leaving a block `}`, all variables declared at the current scope level are unlinked and popped from the hash buckets, ensuring outer shadow variables become visible again in $O(1)$ time.

Topic 26.2: Comprehensive Worked Numerical Problems in Intermediate Code & SDD

🏛️ Solved Numerical 1: Row-Major Address Calculation for 3D Tensor Array

Let A be a 3D array declared as $A[1..10,\ 1..20,\ 1..30]$ of 4-byte integers with base address $\text{base} = 2000$.
Find the exact physical byte address of element $A[4, 12, 18]$ in Row-Major order.

Solution:

$$\text{Bounds: } l_1 = 1, h_1 = 10, \quad l_2 = 1, h_2 = 20, \quad l_3 = 1, h_3 = 30$$

$$\text{Dimension sizes: } n_2 = (20 - 1 + 1) = 20, \quad n_3 = (30 - 1 + 1) = 30$$

$$\text{Offset} = \big[(4 - 1) \times 20 + (12 - 1)\big] \times 30 + (18 - 1)$$

$$\text{Offset} = \big[3 \times 20 + 11\big] \times 30 + 17 = [60 + 11] \times 30 + 17 = 71 \times 30 + 17 = 2130 + 17 = 2147$$

$$\text{Address}(\mathbf{A}[\mathbf{4, 12, 18}]) = \mathbf{2000} + \mathbf{2147} \times \mathbf{4} = \mathbf{2000} + \mathbf{8588} = \mathbf{10588}$$

🏛️ Solved Numerical 2: Column-Major Address Calculation for 2D Array

Let B be a 2D array declared as $B[-5..15,\ 2..10]$ with base address $\text{base} = 5000$ and element width $w = 8$ bytes.
Calculate the exact physical byte address of element $B[3, 7]$ in Column-Major order.

Solution:

$$\text{Bounds: } l_1 = -5, h_1 = 15 \implies n_1 = (15 - (-5) + 1) = 21, \quad l_2 = 2, h_2 = 10$$

$$\text{Column-Major Formula: } \text{Address}(B[i, j]) = \text{base} + \big[(j - l_2) \times n_1 + (i - l_1)\big] \times w$$

$$\text{Offset} = (7 - 2) \times 21 + (3 - (-5)) = 5 \times 21 + 8 = 105 + 8 = 113$$

$$\text{Address}(\mathbf{B}[\mathbf{3, 7}]) = \mathbf{5000} + \mathbf{113} \times \mathbf{8} = \mathbf{5000} + \mathbf{904} = \mathbf{5904}$$

🏛️ Solved Numerical 3: Syntax-Directed Definition for Desktop Arithmetic Calculator

Production	Semantic Rule	Attribute Type
$L \to E\ \mathbf{n}$	$\text{print}(E.\text{val})$	Synthesized
$E \to E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$	Synthesized
$E \to T$	$E.\text{val} = T.\text{val}$	Synthesized
$T \to T_1 * F$	$T.\text{val} = T_1.\text{val} \times F.\text{val}$	Synthesized
$T \to F$	$T.\text{val} = F.\text{val}$	Synthesized
$F \to (E)$	$F.\text{val} = E.\text{val}$	Synthesized
$F \to \mathbf{digit}$	$F.\text{val} = \mathbf{digit}.\text{lexval}$	Synthesized

Q6. Differentiate between Synthesized and Inherited attributes with concrete annotated parse tree examples. (8 Marks)

• **Synthesized Attributes:** Computed strictly from children nodes ($A.s = f(C_1.a, \ldots, C_k.a)$). Evaluated bottom-up in Post-Order traversal (e.g. arithmetic expression values $E.\text{val} = E_1.\text{val} + T.\text{val}$).
• **Inherited Attributes:** Computed from parent node or left siblings ($Y.i = f(\text{Parent}.a, \text{Sibling}.a)$). Evaluated top-down in Pre-Order traversal (e.g. data type propagation in declarations $L.\text{in} = T.\text{type}$).

Q7. What is a Dependency Graph in Syntax-Directed Definitions? How is Topological Sorting used to evaluate attributes? (10 Marks)

A **Dependency Graph** is a directed graph where nodes represent attribute instances at parse tree nodes, and a directed edge from $u \to v$ indicates that attribute v depends on attribute u ($v = f(u, \ldots)$). A valid evaluation order exists if and only if the dependency graph contains zero directed cycles. A **Topological Sort** of the DAG produces a linear evaluation sequence ensuring every attribute is computed before its consumers read it!

Q8. Explain Type Checking and Type Inferences for Expressions with formal typing rules. (8 Marks)

A **Type System** defines rules assigning types to programming constructs. A formal typing rule is expressed as:

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}}$$

Where Γ represents the typing environment (Symbol Table). If operand types differ (e.g. $\text{float} + \text{int}$), the type inference engine checks for valid implicit coercions ($\text{int} \to \text{float}$) before failing with a semantic type error.

Q9. Generate Three-Address Code, Quadruples, Triples, and Indirect Triples for $a = b * -c + b * -c$. (10 Marks)

Three-Address Code:
`t1 = -c`, `t2 = b * t1`, `t3 = -c`, `t4 = b * t3`, `t5 = t2 + t4`, `a = t5`.
Quadruples: `(uminus, c, -, t1)`, `(*, b, t1, t2)`, `(uminus, c, -, t3)`, `(*, b, t3, t4)`, `(+, t2, t4, t5)`, `(=, t5, -, a)`.
Triples: (0): `(uminus, c, -)`, (1): `(*, b, (0))`, (2): `(uminus, c, -)`, (3): `(*, b, (2))`, (4): `(+, (1), (3))`, (5): `(=, a, (4))`.

Q10. Explain the role of Abstract Syntax Trees (AST) vs Concrete Parse Trees in Semantic Analysis. (6 Marks)

A **Concrete Parse Tree** contains every grammatical terminal and non-terminal used during parsing, including punctuation tokens, parentheses, and single-production chains ($E \to T \to F$). An **Abstract Syntax Tree (AST)** compresses the parse tree into an operator-operand hierarchy where operators become interior nodes and operands become leaves, discarding redundant formatting tokens and dramatically accelerating semantic passes.

Topic 26.3: Advanced Type Checking, Equivalence & AST Linearization

A compiler's type checker enforces the type system rules of the source programming language. The type system assigns **Type Expressions** to program elements. A type expression is either a basic type (such as `integer`, `float`, `char`, `type_error`, `void`) or formed by applying a **Type Constructor**:

Array Type Constructor: If T is a type expression, then $\text{array}(I, T)$ is a type expression denoting an array with index set I and elements of type T.
Product Type Constructor: If T_1 and T_2 are type expressions, then their Cartesian product $T_1 \times T_2$ represents pairs of elements.
Record / Structure Constructor: Encapsulates named field elements: $\text{record}((f_1 \times T_1) \times \cdots \times (f_k \times T_k))$.
Function Type Constructor: Maps domain types to range types: $T_1 \to T_2$.
Pointer Constructor: If T is a type, then $\text{pointer}(T)$ represents memory address locations referencing values of type T.

🏛️ Step-by-Step Solved Problem: Type Checking Rules & Structural Equivalence Verification

Consider the following C type definitions:

```
typedef struct { int x; float y; } PointA;
typedef struct { int x; float y; } PointB;
PointA p1;
PointB p2;
```

Evaluation:

- **Under Name Equivalence:** `p1` and `p2` have **DIFFERENT** types because their declared type names (`PointA` vs `PointB`) differ. An assignment `p1 = p2` results in a compile-time static type error!
- **Under Structural Equivalence:** `p1` and `p2` have **IDENTICAL** types because their internal memory structures are both $\text{record}((\text{"x"} \times \text{int}) \times (\text{"y"} \times \text{float}))$. The assignment `p1 = p2` is permitted!

🏛️ Complete Step-by-Step Translation of Array Assignment Statement with Scaling

Source Statement: `A[i][j] = B[i][j] * C[k] + 25.5`

```
// 1. Calculate address of B[i][j] (Dimensions: 10 x 20, element width: 8 bytes)
t1 = i * 20
t2 = t1 + j
t3 = t2 * 8
t4 = B[t3]            ; Load B[i][j] float value into temporary t4

// 2. Calculate address of C[k] (1D Array, element width: 8 bytes)
t5 = k * 8
t6 = C[t5]            ; Load C[k] float value into temporary t6

// 3. Multiply and Add Constant
t7 = t4 * t6          ; Product B[i][j] * C[k]
t8 = t7 + 25.5        ; Add floating-point constant literal

// 4. Calculate target address of A[i][j] (Dimensions: 10 x 20, element width: 8 bytes)
t9 = i * 20
t10 = t9 + j
t11 = t10 * 8
A[t11] = t8           ; Store computed value into A[i][j]
```